



YZM0101

Yapay Zeka ve Makine Öğrenimine

Giriş

Hafta 3

Bilgisiz Arama Stratejileri

Körlemesine Yol Bulmak

- Geçen hafta: Problem formülasyonu ve arama ağaçları.
- Bu hafta: İlk somut arama algoritmalarımız.
- Bilgisiz (Uninformed) Arama: Hedefin nerede olduğuna dair probleme özgü bir bilgi kullanmayan stratejiler.
- Temel Stratejiler: BFS, UCS, DFS.

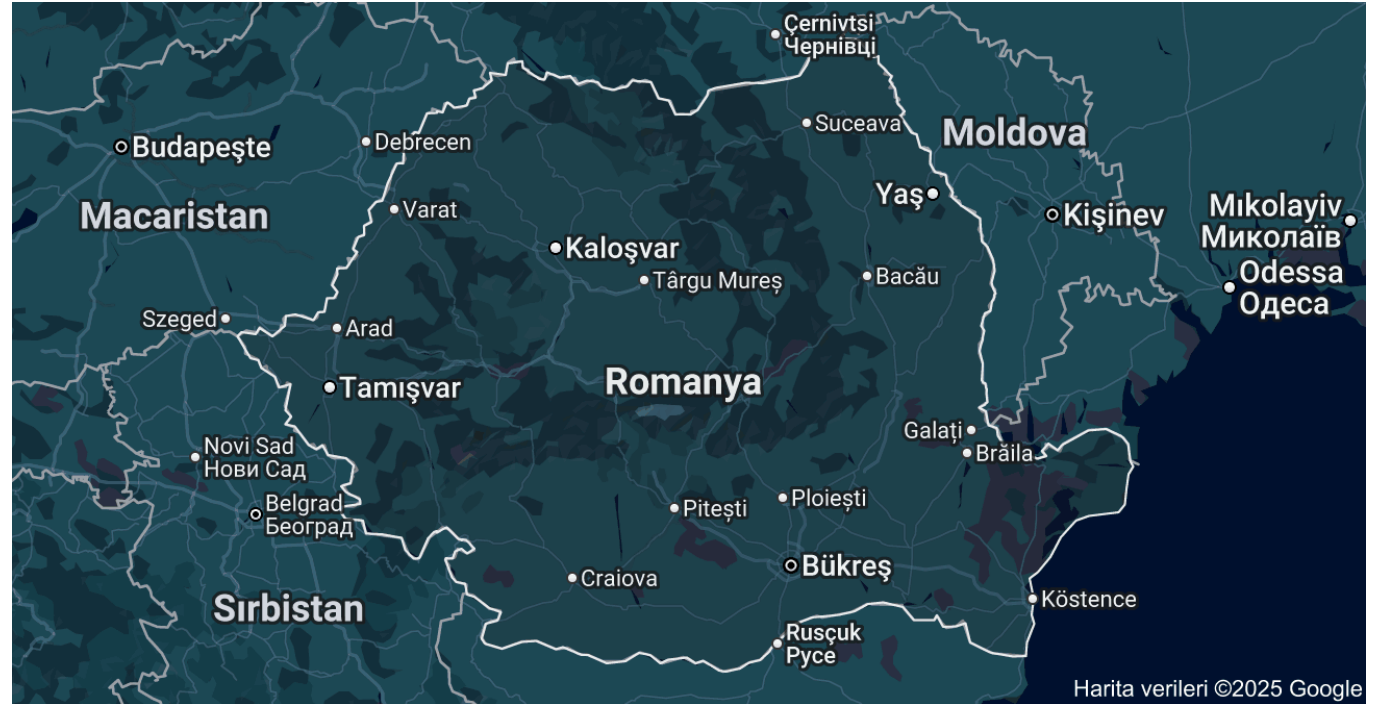
Genişlik Öncelikli Arama (BFS)

Katman Katman Keşif

- Arama ağacının en sığ düğümleri önce genişletilir.
- Kökten başlayarak, seviye seviye (katman katman) arama yapar.
- Frontier (sınır) veri yapısı olarak bir **Kuyruk (Queue - FIFO)** kullanır.
- Yani, en eski keşfedilen düğüm, ilk genişletilen olur.

BFS Algoritması Adım Adım

- 1. **Frontier:** {Arad}
- 2. **Genişlet:** Arad -> ****Frontier:**** {Sibiu, Timisoara, Zerind}
- 3. **Genişlet:** Sibiu -> ****Frontier:**** {Timisoara, Zerind, Fagaras, Oradea, Rimnicu Vilcea}
- 4. **Genişlet:** Timisoara -> ... ve bu şekilde devam eder.



BFS Performans Analizi

Avantajlar ve Dezavantajlar

- **Tamlık (Completeness):** Evet. Eğer en sığ hedef 'd' derinliğindeyse ve dallanma faktörü 'b' sonlu ise, BFS çözümü bulmayı garanti eder.
- **Optimalite (Optimality):** Evet, ancak sadece **tüm aksiyon maliyetleri eşitse**. Çünkü en az adıma sahip yolu bulur, en düşük maliyetli yolu değil.
- **Zaman Karmaşıklığı:** $O(b^d)$. En kötü durumda, 'd' derinliğindeki hedefi bulmadan önce o seviyeye kadar olan tüm düğümleri üretir.
- **Uzay Karmaşıklığı:** $O(b^d)$. En kötü durumda, frontier'da (kuyrukta) bir seviyedeki tüm düğümleri tutması gerekir.

Tekdüze Maliyetli Arama (UCS)

En Ucuz Yolu Önce Keşfet

- Genişletilecek bir sonraki düğümü, kökten olan toplam yol maliyetine ($g(n)$) göre seçer.
- Her zaman en düşük maliyetli yolu genişletir.
- Frontier (sınır) veri yapısı olarak bir ****Öncelikli Kuyruk (Priority Queue)**** kullanır.
- BFS'nin, tüm aksiyon maliyetleri 1 olduğundaki özel bir halidir.

UCS Performans Analizi

Avantajlar ve Dezavantajlar

- **Tamlık (Completeness):** Evet, eğer aksiyon maliyetleri sıfırdan büyükse ($\epsilon > 0$).
- **Optimalite (Optimality):** Evet. UCS, bir hedefi genişlettiğinde, o hedefe giden en düşük maliyetli yolu bulduğunu garanti eder.
- **Zaman Karmaşıklığı:** $O(b^{(1 + \text{floor}(C^*/\epsilon))})$. C^* optimal çözümün maliyeti, ϵ en küçük aksiyon maliyeti.
- **Uzay Karmaşıklığı:** $O(b^{(1 + \text{floor}(C^*/\epsilon))})$. Zaman karmaşıklığı ile aynıdır.

Derinlik Öncelikli Arama (DFS)

Bir Yolu Sonuna Kadar Takip Et

- Arama ağacının en derin düğümleri önce genişletilir.
- Bir yolu, daha fazla ilerleyemeyene kadar takip eder, sonra geri döner (backtracking) ve başka bir yolu dener.
- Frontier (sınır) veri yapısı olarak bir **Yığın (Stack - LIFO)** kullanılır.
- Yani, en son keşfedilen düğüm, ilk genişletilen olur.

DFS Performans Analizi

Avantajlar ve Dezavantajlar

- **Tamlık (Completeness):** Hayır. Döngüler içeren (sonsuz) arama uzaylarında, sonsuz bir yola saplanıp kalabilir. Sonlu uzaylarda tamdır.
- **Optimalite (Optimality):** Hayır. Genellikle bulduğu ilk çözüm, optimal çözüm değildir. Daha derinde ama daha kötü bir çözümü, daha sığdaki iyi bir çözümden önce bulabilir.
- **Zaman Karmaşıklığı:** $O(b^m)$. En kötü durumda, durum uzayının en derin noktasına (m) kadar olan tüm düğümleri ziyaret edebilir.
- **Uzay Karmaşıklığı:** $O(b \cdot m)$. Bu, DFS'nin en büyük avantajıdır. Hafızada sadece mevcut yolu ve o yoldaki her düğümün keşfedilmemiş kardeşlerini tutması yeterlidir.

Algoritmaların Karşılaştırılması

Hangi Durumda Hangisi?

- **BFS:** Çözümün sığda olduğundan eminseniz ve tüm adım maliyetleri eşitse kullanılır. Optimal ve tamdır ama hafıza kullanımı yüksektir.
- **UCS:** Adım maliyetleri farklıysa ve optimal çözüm garantisi isteniyorsa kullanılır. Tam ve optimaldir ama hedefin nerede olduğunu bilmez.
- **DFS:** Arama uzayı çok genişse, hafıza kısıtlıysa ve çözümün çok derinlerde olduğu düşünülüyorsa kullanılır. Hafıza dostudur ama tam ve optimal değildir.

Derinlik-Sınırlı Arama (DLS)

DFS'nin Sonsuz Döngü Sorununa Çözüm

- Depth-Limited Search (DLS), DFS ile aynıdır ancak önceden belirlenmiş bir 'derinlik limiti' (limit) vardır.
- Arama, bu limite ulaştığında o yolu daha fazla keşfetmez ve geri döner.
- Bu, DFS'nin sonsuz yollara saplanmasını engeller.
- ****Problem:**** Doğru derinlik limitini nasıl bileceğiz? Eğer $\text{limit} < d$ ise, DLS çözümü bulamaz (tam değil). Eğer $\text{limit} > d$ ise, optimal olmayabilir.

Yinelemeli Derinleştirme (IDS)

BFS ve DFS'nin En İyi Yanlarını Birleştirmek

- Iterative Deepening Search (IDS), DLS'yi tekrar tekrar farklı limitlerle çalıştırır.
- Limit = 0 için DLS çalıştır, çözüm yoksa;
- Limit = 1 için DLS çalıştır, çözüm yoksa;
- Limit = 2 için DLS çalıştır, ... ve bu şekilde devam eder.
- BFS'nin tamlık ve optimalitesini (adım maliyetleri eşitse), DFS'nin düşük hafıza kullanımıyla birleştirir.

IDS Performans Analizi

Neden Verimsiz Değil?

- **Tamlık (Completeness):** Evet.
- **Optimalite (Optimality):** Evet, eğer adım maliyetleri eşitse (BFS gibi).
- **Zaman Karmaşıklığı:** $O(b^d)$. BFS ile aynı mertebededir.
- **Uzay Karmaşıklığı:** $O(b \cdot d)$. DFS ile aynı mertebededir.

Çift Yönlü Arama (Bidirectional Search)

Ortada Buluşalım

- Aynı anda hem başlangıç durumundan ileriye doğru, hem de hedef durumundan geriye doğru iki arama başlatır.
- İki arama ağacı ortada bir yerde karşılaştığında arama sona erer.
- **Fikir:** İki küçük ağaç keşfetmek, bir büyük ağaç keşfetmekten çok daha verimlidir.
- Zaman/Uzay Karmaşıklığı: $O(b^{(d/2)})$.
- Durumunun tek ve net olarak bilinmesi gerekir ve aksiyonların tersine çevrilebilir olması gerekir

Bilgisiz Aramanın Sınırları

Neden Daha İyisine İhtiyacımız Var?

- Bilgisiz arama algoritmaları, durum uzayını körlemesine tarar.
- Dallanma faktörü biraz bile büyük olduğunda, bu algoritmaların karmaşıklığı (özellikle BFS ve UCS) pratik limitleri aşar.
- Örnek: $b=10$, $d=12$ olan bir problemde BFS, 1 trilyondan fazla düğüm üretir ve 10 terabayttan fazla hafızaya ihtiyaç duyar.
- Çözüm: Probleme özgü bilgiler kullanarak aramaya 'rehberlik etmek'.

Uygulama Notları: Frontier Veri Yapıları

Python'da Nasıl Kodlanır?

- **Kuyruk (Queue) - BFS için:**

- - Python'un ``collections.deque`` objesi verimli bir kuyruk olarak kullanılabilir.
- - ``append()`` ile sona ekleme, ``popleft()`` ile baştan çıkarma.

- **Yığın (Stack) - DFS için:**

- - Python'un standart ``list`` objesi bir yığın gibi davranabilir.
- - ``append()`` ile sona (üste) ekleme, ``pop()`` ile sondan (üstten) çıkarma.

- **Öncelikli Kuyruk (Priority Queue) - UCS için:**

- - Python'un ``heapq`` modülü, bir liste üzerinde min-heap (en küçük eleman her zaman tepededir) yapısı kurar.
- - ``heappush()`` ile ekleme, ``heappop()`` ile en küçük elemanı çıkarma.

Bilgisiz Arama Algoritmaları

Kriter	BFS	UCS	DFS	IDS
Tamlık	Evet	Evet	Hayır	Evet
Optimalite	Evet*	Evet	Hayır	Evet*
Zaman	$O(b^d)$	$O(b^{(1+C^*/\epsilon)})$	$O(b^m)$	$O(b^d)$
Uzay	$O(b^d)$	$O(b^{(1+C^*/\epsilon)})$	$O(b^*m)$	$O(b^*d)$

Haftanın Kavramları: Özet

- Akılda Kalması Gerekenler

- **BFS (Genişlik Öncelikli):** Kuyruk (FIFO) kullanır, sığ ve en az adımlı çözümü bulur. Hafıza kullanımı yüksektir.
- **UCS (Tekdüze Maliyetli):** Öncelikli Kuyruk kullanır, en düşük maliyetli çözümü bulur. Optimaldir.
- **DFS (Derinlik Öncelikli):** Yığın (LIFO) kullanır, bir yolu sonuna kadar gider. Hafıza kullanımı düşüktür, ama tam ve optimal değildir.
- **IDS (Yinelemeli Derinleştirme):** BFS'nin garantilerini DFS'nin hafıza verimliliği ile birleştirir.

Gelecek Hafta: Bilgili Arama Stratejileri

- Aramaya nasıl 'rehberlik' ederiz?
- Sezgisel (Heuristic) Fonksiyon nedir? ($h(n)$)
- Açgözlü En İyi Öncelikli Arama (Greedy Best-First Search)
- A* Arama: BFS, UCS ve Greedy Search'ün birleşimi.
- Kabul edilebilir (admissible) ve tutarlı (consistent) sezgiseller.

Düşünme Sorusu

- BFS vs. UCS
 - Romanya haritasında, Arad'dan Bükreş'e gitmek istiyoruz.
 - BFS, {Arad, Sibiu, Fagaras, Bucharest} yolunu (3 adım) bulur.
 - UCS, {Arad, Sibiu, Rimnicu Vilcea, Pitesti, Bucharest} yolunu (4 adım) bulur.
 - Neden UCS daha uzun bir yol bulmasına rağmen 'optimal' algoritma olarak kabul edilir?
 - Bu iki algoritmanın 'optimalite' tanımları arasındaki fark nedir?

Ödev İpucu

- Arama Algoritmalarını Kodlarken
 - ****Modüler Düşünün:**** Problemi okuyan, aksiyonları üreten, hedef testi yapan fonksiyonları ayrı ayrı yazın.
 - ****Düğüm Sınıfı (Node Class):**** Durum, ata, aksiyon ve maliyet bilgilerini bir arada tutan bir 'Node' sınıfı veya yapısı oluşturmak işinizi çok kolaylaştırır.
 - ****Veri Yapılarına Dikkat:**** BFS için `deque`, DFS için `list`, UCS için `heapq` kullandığınızdan emin olun.
 - ****Test Edin:**** Basit, çözümü bilinen küçük bir graf üzerinde algoritmanızın doğru çalışıp çalışmadığını test ederek başlayın.

Gelecek Adımlar

- Bilgisiz Aramanın Ötesi

- Bu hafta öğrendiğimiz algoritmalar, arama teorisinin temelidir.
- Ancak gerçek dünya problemlerinin karmaşıklığı, genellikle daha 'akıllı' yaklaşımlar gerektirir.
- Gelecek hafta, aramaya sezgisel bilgiler ekleyerek bu algoritmaları nasıl binlerce kat hızlandırabileceğimizi göreceğiz.
- Bu, 'bilgi'nin hesaplama maliyetini nasıl düşürdüğünün harika bir örneği olacak.



• Sorular